Copy-on-Write (CoW)

> **Note**
>
> Copy-on-Write is now the default with pandas 3.0.

Copy-on-Write was first introduced in version 1.5.0. Starting from version 2.0 most of the optimizations that become possible through CoW are implemented and supported. All possible optimizations are supported starting from pandas 2.1.

CoW will lead to more predictable behavior since it is not possible to update more than one object with one statement, e.g. indexing operations or methods won't have side-effects. Additionally, through delaying copies as long as possible, the average performance and memory usage will improve.

Previous behavior

pandas indexing behavior is tricky to understand. Some operations return views while other return copies. Depending on the result of the operation, mutating one object might accidentally mutate another:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

Mutating `subset`, e.g. updating its values, also updated `df`. The exact behavior was hard to predict. Copy-on-Write solves accidentally modifying more than one object, it explicitly disallows this. `df` is unchanged:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [2]: subset = df["foo"]

In [3]: subset.iloc[0] = 100

In [4]: df
Out[4]:
   foo  bar
0    1    4
1    2    5
2    3    6
```

The following sections will explain what this means and how it impacts existing applications.

Migrating to Copy-on-Write

Copy-on-Write is the default and only mode in pandas 3.0. This means that users need to migrate their code to be compliant with CoW rules.

The default mode in pandas < 3.0 raises warnings for certain cases that will actively change behavior and thus change user intended behavior.

pandas 2.2 has a warning mode

```
pd.options.mode.copy_on_write = "warn"
```

that will warn for every operation that will change behavior with CoW. We expect this mode to be very noisy, since many cases that we don't expect that they will influence users will also emit a warning. We recommend checking this mode and analyzing the warnings, but it is not necessary to address all of these warning. The first two items of the following lists are the only cases that need to be addressed to make existing code work with CoW.

The following few items describe the user visible changes:

Chained assignment will never work

`loc` should be used as an alternative. Check the chained assignment section for more details.

Accessing the underlying array of a pandas object will return a read-only view

```
In [5]: ser = pd.Series([1, 2, 3])

In [6]: ser.to_numpy()
Out[6]: array([1, 2, 3])
```

This example returns a NumPy array that is a view of the Series object. This view can be modified and thus also modify the pandas object. This is not compliant with CoW rules. The returned array is set to non-writeable to protect against this behavior. Creating a copy of this array allows modification. You can also make the array writeable again if you don't care about the pandas object anymore.

See the section about read-only NumPy arrays for more details.

Only one pandas object is updated at once

The following code snippet updated both `df` and `subset` without CoW:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

This is not possible anymore with CoW, since the CoW rules explicitly forbid this. This includes updating a single column as a `Series` and relying on the change propagating back to the parent `DataFrame`. This statement can be rewritten into a single statement with `loc` or `iloc` if this behavior is necessary. `DataFrame.where()` is another suitable alternative for this case.

Updating a column selected from a `DataFrame` with an inplace method will also not work anymore.

```
In [7]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [8]: df["foo"].replace(1, 5, inplace=True)
Out[8]:
0    5
1    2
2    3
Name: foo, dtype: int64

In [9]: df
Out[9]:
   foo  bar
0    1    4
1    2    5
2    3    6
```

This is another form of chained assignment. This can generally be rewritten in 2 different forms:

```
In [10]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [11]: df.replace({"foo": {1: 5}}, inplace=True)
Out[11]:
   foo  bar
0    5    4
1    2    5
2    3    6

In [12]: df
Out[12]:
   foo  bar
0    5    4
1    2    5
2    3    6
```

A different alternative would be to not use `inplace`:

```
In [13]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [14]: df["foo"] = df["foo"].replace(1, 5)

In [15]: df
Out[15]:
   foo  bar
0    5    4
1    2    5
2    3    6
```

Constructors now copy NumPy arrays by default

The Series and DataFrame constructors now copies a NumPy array by default when not otherwise specified. This was changed to avoid mutating a pandas object when the NumPy array is changed

inplace outside of pandas. You can set `copy=False` to avoid this copy.

Description

CoW means that any DataFrame or Series derived from another in any way always behaves as a copy. As a consequence, we can only change the values of an object through modifying the object itself. CoW disallows updating a DataFrame or a Series that shares data with another DataFrame or Series object inplace.

This avoids side-effects when modifying values and hence, most methods can avoid actually copying the data and only trigger a copy when necessary.

The following example will operate inplace:

```
In [16]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [17]: df.iloc[0, 0] = 100

In [18]: df
Out[18]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

The object `df` does not share any data with any other object and hence no copy is triggered when updating the values. In contrast, the following operation triggers a copy of the data under CoW:

```
In [19]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [20]: df2 = df.reset_index(drop=True)

In [21]: df2.iloc[0, 0] = 100

In [22]: df
Out[22]:
   foo  bar
0    1    4
1    2    5
2    3    6

In [23]: df2
Out[23]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

`reset_index` returns a lazy copy with CoW while it copies the data without CoW. Since both objects, `df` and `df2` share the same data, a copy is triggered when modifying `df2`. The object `df` still has the same values as initially while `df2` was modified.

If the object `df` isn't needed anymore after performing the `reset_index` operation, you can emulate an inplace-like operation through assigning the output of `reset_index` to the same variable:

```
In [24]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [25]: df = df.reset_index(drop=True)

In [26]: df.iloc[0, 0] = 100

In [27]: df
Out[27]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

The initial object gets out of scope as soon as the result of `reset_index` is reassigned and hence `df` does not share data with any other object. No copy is necessary when modifying the object. This is generally true for all methods listed in Copy-on-Write optimizations.

Previously, when operating on views, the view and the parent object was modified:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

CoW triggers a copy when `df` is changed to avoid mutating `view` as well:

```
In [28]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [29]: view = df[:]

In [30]: df.iloc[0, 0] = 100

In [31]: df
Out[31]:
   foo  bar
0  100    4
1    2    5
2    3    6

In [32]: view
Out[32]:
   foo  bar
0    1    4
1    2    5
2    3    6
```

Chained Assignment

Chained assignment references a technique where an object is updated through two subsequent indexing operations, e.g.

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: df["foo"][df["bar"] > 5] = 100
In [3]: df
Out[3]:
   foo  bar
0    1    4
1    2    5
2  100    6
```

The column `foo` was updated where the column `bar` is greater than 5. This violated the CoW principles though, because it would have to modify the view `df["foo"]` and `df` in one step.

Hence, chained assignment will consistently never work and raise a `ChainedAssignmentError` warning with CoW enabled:

```
In [33]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})

In [34]: df["foo"][df["bar"] > 5] = 100
```

With copy on write this can be done by using `loc`.

```
In [35]: df.loc[df["bar"] > 5, "foo"] = 100
```

Read-only NumPy arrays

Accessing the underlying NumPy array of a DataFrame will return a read-only array if the array shares data with the initial DataFrame:

The array is a copy if the initial DataFrame consists of more than one array:

```
In [36]: df = pd.DataFrame({"a": [1, 2], "b": [1.5, 2.5]})

In [37]: df.to_numpy()
Out[37]:
array([[1. , 1.5],
       [2. , 2.5]])
```

The array shares data with the DataFrame if the DataFrame consists of only one NumPy array:

```
In [38]: df = pd.DataFrame({"a": [1, 2], "b": [3, 4]})

In [39]: df.to_numpy()
Out[39]:
array([[1, 3],
       [2, 4]])
```

This array is read-only, which means that it can’t be modified inplace:

```
In [40]: arr = df.to_numpy()

In [41]: arr[0, 0] = 100
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[41], line 1
----> 1 arr[0, 0] = 100

ValueError: assignment destination is read-only
```

The same holds true for a Series, since a Series always consists of a single array.

There are two potential solutions to this:

- Trigger a copy manually if you want to avoid updating DataFrames that share memory with your array.
- Make the array writeable. This is a more performant solution but circumvents Copy-on-Write rules, so it should be used with caution.

```
In [42]: arr = df.to_numpy()
```

© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud.

Built with the PyData Sphinx Theme 0.16.1.

Created using Sphinx 9.0.4.